

AETHERIA — Function / Module List

Step 2 · maps modules → public functions for Step 4 coding plan

A. Auth & Account

- `auth.signupWithEmail({email, password, displayName})`
- `auth.loginWithEmail({email, password})`
- `auth.loginWithOAuth(provider, code)`
- `auth.refreshToken(refreshToken)`
- `auth.logout()`
- `auth.requestPasswordReset(email)`
- `auth.completePasswordReset(token, newPassword)`
- `auth.verifyEmail(token)`
- `account.getProfile(userId)`
- `account.updateProfile(userId, patch)`
- `account.deleteAccount(userId) // GDPR`

B. Roster & Skills

- `roster.list(userId)`
- `roster.unlockCharacter(userId, characterId)`
- `roster.ascend(userId, characterId)`
- `roster.equipSkin(userId, characterId, skinId)`
- `skills.tree(characterId)`
- `skills.invest(userId, skillId, points)`
- `skills.respec(userId, characterId)`

C. World / Levels

- `world.realms()` // catalog
- `world.levelsForRealm(realmId)`
- `world.startLevel(userId, levelId) → returns runId`
- `world.resumeRun(runId)`
- `world.abandonRun(runId)`

D. Combat Engine (packages/domain-combat)

- `combat.createBattle(seed, mapDef, partyDef, encounterDef)`
- `combat.applyAction(state, action) → { nextState, events[] }`
- `combat.endTurn(state) → resolves status effects, AP refresh`

- `combat.checkVictory(state)`
- `combat.serializeState(state) / combat.hydrate(payload)`
- helpers: `apCost`, `lineOfSight`, `rangeReachable`, `elementAdvantage`, `resonanceCheck`

E. Save / Resume

- `save.snapshot(userId, slot, payload)`
- `save.list(userId)`
- `save.load(userId, slot)`
- `save.delete(userId, slot)`
- `save.autosaveTick(userId, payload) // throttled, slot=0`
- `save.reconcile(local, remote) // conflict resolver`

F. Progression / Level-up

- `progression.grantXp(userId, characterId, xp, source)`
- `progression.checkLevelUp(userId, characterId) → emits LevelledUp`
- `progression.unlockReward(userId, milestone)`
- `progression.accountXp(userId, xp, source)`

G. Inventory & Items

- `inventory.list(userId)`
- `inventory.grant(userId, itemId, qty, source)`
- `inventory.consume(userId, itemId, qty)`
- `inventory.equip(userId, characterId, itemId, slot)`
- `inventory.craft(userId, recipeId)`

H. Quests & Battle Pass

- `quests.dailyForUser(userId)`
- `quests.weeklyForUser(userId)`
- `quests.progress(event) // event-driven`
- `quests.claim(userId, questId)`
- `bp.currentSeason()`
- `bp.progress(userId) → tier, xp`
- `bp.claim(userId, tier)`

I. Marketplace

- `shop.catalog()`
- `shop.purchase(userId, shopItemId, qty)`

- `shop.history(userId)`

J. Social: Guilds / Friends / Chat

- `guild.create(userId, name, tag)`
- `guild.invite(guildId, targetUserId)`
- `guild.respond(invitationId, accept)`
- `guild.kick(guildId, actorId, targetUserId)`
- `guild.promote(guildId, actorId, targetUserId, role)`
- `guild.startRaid(guildId, raidId)`
- `friends.list(userId)`
- `friends.request(userId, targetUserId)`
- `friends.respond(requestId, accept)`
- `chat.send(channel, senderId, content)`
- `chat.history(channel, before, limit)`
- `chat.report(messageId, reason)`

K. PvP & Matchmaking

- `pvp.queue(userId, mode)`
- `pvp.cancelQueue(userId)`
- `pvp.match(matchId) // current state`
- `pvp.submitAction(matchId, userId, action) // server validates`
- `mmr.get(userId, mode)`
- `mmr.update(matchId) // post-match`

L. Leaderboards

- `lb.top(mode, region, n)`
- `lb.aroundUser(userId, mode, n)`
- internal: `lb.recordResult(userId, mode, mmr)` (writes Redis ZSET)

M. Notifications

- `notify.list(userId, unreadOnly)`
- `notify.markRead(userId, ids)`
- internal: `notify.push(userId, type, payload)`

N. Admin / Ops

- `admin.banUser(userId, reason)`
- `admin.grantItem(userId, itemId, qty, reason) // logged in audit`

- `admin.featureFlag.set(key, value)`
- `admin.replay(runId)`

O. Cron / Worker Jobs

- `cron.dailyReset` 00:00 UTC → resets daily quests, login bonus
- `cron.weeklyReset` Mon 00:00 UTC
- `cron.seasonReset` per season config
- `worker.antiCheatScan` per run completion
- `worker.leaderboardSnapshot` hourly
- `worker.guildRaidScheduler` weekly

P. Telemetry / Analytics

- `telemetry.event(name, props, userId)` (client + server)
- internal funnels: signup → tutorial-complete → level-5 → level-25

Cross-cutting concerns

- `errors.AppError`, typed error codes for tRPC
- `audit.write(action, target, payload, ip)`
- `featureFlag.isOn(key, userId)`
- `i18n.t(key, vars)`